

1. Create the C file

nano process_sync.c

2. Write the program using `wait()`

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int i, status;
    pid_t pid;

    printf("Parent Process Started\n");
    printf("Parent PID: %d\n\n", getpid());

    for (i = 1; i <= 3; i++)
    {
        pid = fork();

        if (pid < 0)
        {
            perror("fork failed");
            exit(1);
        }

        if (pid == 0)
        {
            printf("Child %d started | PID = %d | PPID = %d\n",
                i, getpid(), getppid());

            sleep(i);

            printf("Child %d completed\n", i);
            exit(i);
        }
    }

    printf("\nParent is waiting for children...\n");

    for (i = 0; i < 3; i++)
    {
        pid = wait(&status);

        if (WIFEXITED(status))
        {
            printf("Parent collected Child PID %d, Exit Status = %d\n",
                pid, WEXITSTATUS(status));
        }
    }

    printf("\nAll child processes completed.\n");
    printf("Parent process terminating.\n");

    return 0;
}
```

3. Compile the program

```
adithya@LAPTOP-07A78KDI:~$ nano process_sync.c
adithya@LAPTOP-07A78KDI:~$ gcc process_sync.c -o process_sync
adithya@LAPTOP-07A78KDI:~$ ./process_sync
Parent Process Started
Parent PID: 573

Child 1 started | PID = 574 | PPID = 573
Child 2 started | PID = 575 | PPID = 573

Parent is waiting for children...
Child 3 started | PID = 576 | PPID = 573
Child 1 completed
Parent collected Child PID 574, Exit Status = 1
Child 2 completed
Parent collected Child PID 575, Exit Status = 2
Child 3 completed
Parent collected Child PID 576, Exit Status = 3

All child processes completed.
Parent process terminating.
adithya@LAPTOP-07A78KDI:~$ |
```

4. Create another file:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int i, status;
    pid_t children[3];

    printf("Parent PID: %d\n\n", getpid());

    for (i = 0; i < 3; i++)
    {
        children[i] = fork();

        if (children[i] < 0)
        {
            perror("fork failed");
            exit(1);
        }

        if (children[i] == 0)
        {
            printf("Child %d started | PID = %d\n",
                i + 1, getpid());

            sleep((3 - i) * 2);

            printf("Child %d completed\n", i + 1);
            exit(i + 10);
        }
    }

    printf("\nParent waiting for specific children...\n");

    for (i = 0; i < 3; i++)
    {
        pid_t pid;

        pid = waitpid(children[i], &status, 0);

        if (pid > 0 && WIFEXITED(status))
        {
            printf("Parent collected Child PID %d | Exit Status = %d\n",
                pid, WEXITSTATUS(status));
        }
    }

    printf("\nAll children completed.\n");

    return 0;
}
```

5. Compile The Program

```
adithya@LAPTOP-07A78KDI:~$ nano waitpid_example.c
adithya@LAPTOP-07A78KDI:~$ gcc waitpid_example.c -o waitpid_example
adithya@LAPTOP-07A78KDI:~$ ./waitpid_example
Parent PID: 663

Child 1 started | PID = 664
Child 2 started | PID = 665

Parent waiting for specific children...
Child 3 started | PID = 666
Child 3 completed
Child 2 completed
Child 1 completed
Parent collected Child PID 664 | Exit Status = 10
Parent collected Child PID 665 | Exit Status = 11
Parent collected Child PID 666 | Exit Status = 12

All children completed.
adithya@LAPTOP-07A78KDI:~$ |
```

6. Zombie

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0)
    {
        printf("Child: PID = %d\n", getpid());
        printf("Child: Exiting now...\n");

        exit(0);
    }
    else
    {
        printf("Parent: PID = %d\n", getpid());
        printf("Parent: Child PID = %d\n", pid);

        printf("Parent: Sleeping for 30 seconds without wait()...\n");

        sleep(30);

        printf("Parent: Finished.\n");
    }

    return 0;
}
```

Compile

```
adithya@LAPTOP-07A78KDI:~$ nano zombie.c
adithya@LAPTOP-07A78KDI:~$ gcc zombie.c -o zombie
adithya@LAPTOP-07A78KDI:~$ ./zombie
Parent: PID = 725
Parent: Child PID = 726
Parent: Sleeping for 30 seconds without wait()...
Child: PID = 726
Child: Exiting now...
Parent: Finished.
```

```
adithya@LAPTOP-07A78KDI:~$ ps -ef | grep zombie
adithya      739      343  0 07:03 pts/0    00:00:00 grep --color=auto zombie
```

Create a new file

Nano zombie_fixed.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;
    int status;

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0)
    {
        printf("Child PID = %d\n", getpid());
        sleep(2);
        printf("Child terminating...\n");
        exit(25);
    }
    else
    {
        printf("Parent PID = %d\n", getpid());
        printf("Parent waiting for child...\n");
        wait(&status);

        if (WIFEXITED(status))
        {
            printf("Child terminated normally\n");
            printf("Exit status = %d\n",
                   WEXITSTATUS(status));
        }

        printf("Zombie removed.\n");
    }

    return 0;
}
```

Compile The Program

```
adithya@LAPTOP-07A78KDI:~$ nano zombie_fixed.c
adithya@LAPTOP-07A78KDI:~$ gcc zombie_fixed.c -o zombie_fixed
adithya@LAPTOP-07A78KDI:~$ ./zombie_fixed
Parent PID = 816
Parent waiting for child...
Child PID = 817
Child terminating...
Child terminated normally
Exit status = 25
Zombie removed.
adithya@LAPTOP-07A78KDI:~$ |
```

```
adithya@LAPTOP-07A78KDI:~$ ps -el | grep Z
F S    UID      PID     PPID  C  PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
adithya@LAPTOP-07A78KDI:~$ |
```